

크롤링 데이터 & Contract 구조

ABC마트 · 29CM 두 판매처에서 지금 실제로 수집하는 필드와, 그 데이터를 검증·비교하는 Contract(JSON Schema) 구조를 정리했다.

BRANCH

sandbox-python-crawler/jw

범위

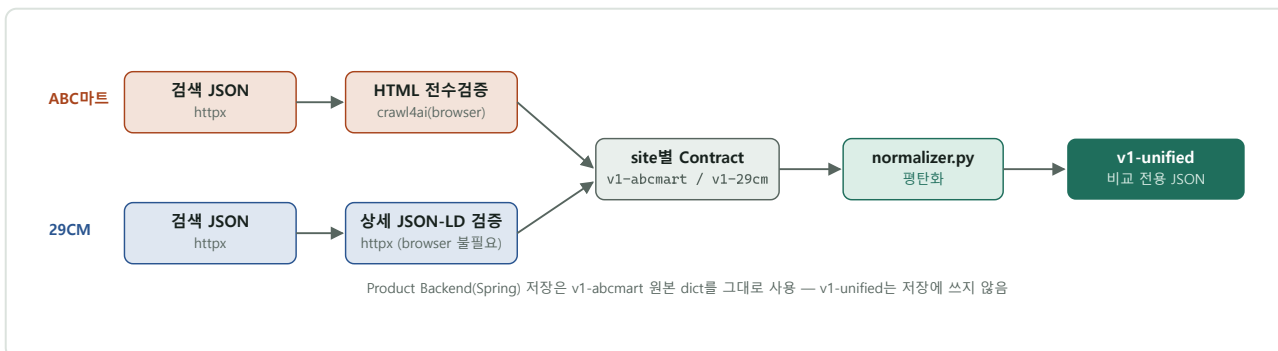
purchase-research-agent (FastAPI/Python)

기준일

2026-08-05

1. 한눈에 보기

두 판매처 크롤러는 각자의 원본 구조(v1-abcmart / v1-29cm)로 상품을 반환한다. `contract_validation.py`가 site별 스키마로 그 자리에서 걸러내고, 비교가 필요할 때만 `normalizer.py`가 하나의 통일 구조(v1-unified)로 평탄화한다.



검색 단계부터 갈라진다 — ABC마트는 검증에도 브라우저가 필요하고, 29CM은 끝까지 순수 HTTP로 끝난다. (2절·3절 참조)

2. ABC마트 vs 29CM — 수집 방식 비교

● ABCMART

AbcMartCrawler

검색 데이터 원천

검색 JSON (기본값)

검증 방법

같은 검색 페이지 HTML 전수 대조

브라우저 필요

필요 — 검색 화면이 JS 렌더링(SPA)

카테고리 크롤링

지원 (17개 카테고리)

상세 수집 방식

병렬 (arun_many + gather)

● 29CM

Cm29Crawler

검색 데이터 원천

검색 JSON (유일한 원천)

검증 방법

상품 상세 HTML의 JSON-LD 대조

브라우저 필요

불필요 — 상세 HTML에 JSON-LD가 그대로 있음

카테고리 크롤링

미구현 (키워드 검색만)

상세 수집 방식

순차 (상품당 0.5초 딜레이)

왜 ABC마트만 브라우저가 필요한가 — ABC마트 검색/카테고리 페이지는 JS로 상품 카드를 렌더링하는 SPA라서 순수 HTTP GET으로는 상품이 안 보인다. 그래서 검증 단계에만 Crawl4AI(Playwright)를 쓴다. 29CM은 상세 페이지의 SEO용 Product JSON-LD가 렌더링 전 HTML에 그대로 박혀 있어서 검색·검증 전 구간을 httpx만으로 끝낼 수 있다.

3. ABC마트 — 수집 필드 `contracts/collector/v1-abcmart`

SEARCH

검색 단계 — `AbcJsonFetcher + AbcMartCrawler._parse`

필드	타입	설명
<code>source_product_id</code>	<code>string</code>	PRDT_NO. product.site + source_product_id가 DB UNIQUE KEY
<code>title / brand</code>	<code>string</code>	PRDT_NAME / BRAND_NAME
<code>price / price_original</code>	<code>string</code>	"19,000원" 형태 문자열 (PRDT_DC_PRICE / NRML_AMT)
<code>discount_percent</code>	<code>int null</code>	DISCOUNT_RATE
<code>image_url / color / style_code</code>	<code>string</code>	PRDT_IMAGE_URL / COLOR_ID / STYLE_INFO
<code>link</code>	<code>string</code>	prdtNo 기반 상세 URL, source_product_id와 일치해야 함
<code>review_count</code>	<code>int</code>	RVW_COUNT (검색 JSON에 이미 포함)
<code>options.colors / .sizes</code>	<code>string[]</code>	COLOR_ID / PRDT_OPTION-SIZE_LIST — 검색 JSON에서 바로 채워짐

DETAIL 상세 단계 — `DetailFetcher.attach()` (병렬)

상위 `detail_limit`개(전체 지정 가능) 상품에 대해 상세 페이지를 `crawl4ai` `arun_many`로 한 번에 렌더링하고, 리뷰·옵션 API를 `asyncio.gather`로 동시 호출한다.

필드	타입	설명
<code>images</code>	<code>string[]</code>	상세 페이지 메인 슬라이더 이미지 (shrink=580:580)
<code>category / category_path</code>	<code>string</code>	상세 페이지 내 category JSON에서 추출
<code>in_stock</code>	<code>bool null</code>	sold-out 아닌 사이즈 버튼이 하나라도 있으면 true
<code>rating</code>	<code>number null</code>	reviews[].score 평균값으로 계산 (리뷰 없으면 null)
<code>reviews[]</code>	<code>object[]</code>	전량 페이지네이션(50개/페이지) 수집. content, score, detail_scores(착화감·가격·디자인·내구성·경량성), color, helpful_count, is_best, images

VERIFY 검증 — `verification.py: reconcile_page()`

같은 페이지·검색어의 검색 JSON과 렌더링 HTML을 상품ID 기준으로 연결해 `title`, `brand`, `price`, `price_original`, `discount_percent`, `image_url`, `link` 6개 필드를 대조한다.

STATUS	의미
MATCHED	JSON과 HTML 값이 모두 일치
MISMATCH	하나 이상의 필드가 다름 — differences[]에 필드별 json_value/html_value 기록
MISSING_IN_HTML	JSON엔 있는데 렌더링 화면엔 없는 상품
FAILED	해당 페이지 브라우저 로드 자체가 실패

4. 29CM — 수집 필드 `contracts/collector/v1-29cm`

SEARCH 검색 단계 — `Cm29Crawler._parse`

필드	타입	설명
<code>source_product_id</code>	<code>string</code>	itemId를 문자열로 변환
<code>title / brand</code>	<code>string</code>	itemInfo.productName / brandName
<code>price / price_original</code>	<code>string</code>	itemInfo.sellPrice / originalPrice → "원" 문자열 변환
<code>discount_percent</code>	<code>int null</code>	itemInfo.saleRate 반올림
<code>image_url / link</code>	<code>string</code>	thumbnailUrl / itemUrl.webLink
<code>color / style_code</code>	<code>string</code>	검색 API엔 없음 — 상세 수집 전까지 항상 빈 문자열

DETAIL 상세 단계 — `Cm29DetailFetcher.attach()` (순차)

상품별 상세 HTML(JSON-LD + 스트리밍 옵션 데이터) GET과 리뷰 API GET을 상품당 0.5초 딜레이로 순차 호출한다.

필드	타입	설명
images	string[]	JSON-LD image[] — 상세컷 포함
category / category_path	string	JSON-LD category + BreadcrumbList
category_codes	object	categoryLarge/Medium/SmallCode
in_stock	bool null	JSON-LD offers.availability에 "InStock" 포함 여부
options[]	{name,value}[]	__next_f 스트리밍 데이터 정규식 파싱 (SIZE-COLOR, KR/IT 사이즈 혼재)
color	string	options[].value에서 색상만 추출해 콤마로 합침
rating / review_count	number/int null	리뷰 API averagePoint/count 우선, 없으면 JSON-LD aggregateRating
reviews[]	object[]	전량 페이지네이션(50개/페이지) 수집. user_size, partner_comment, size_survey(작음/맞음/큼), is_blind 포함

VERIFY 검증 — `verification.py: compare_product()`

검색 JSON 상품마다 상세 HTML을 요청해 SEO용 Product JSON-LD를 파싱하고, ABC마트와 동일한 6개 필드를 대조한다. 상품 1건당 요청 1회가 필요해 ABC마트의 페이지 단위 검증보다 요청 수가 많다.

5. Contract 구조 contracts/collector/

스키마는 세 계층으로 나뉜다. site별 원본 계약이 `additionalProperties: false`로 크롤러 출력을 그 자리에서 강제하고, 비교가 필요할 때만 `normalizer.py`를 거쳐 통일 계약으로 변환한다. 운영 저장 계약(Go Collector 용)은 이 둘과 완전히 별개다.

계층	위치	강제 시점	용도
v1-abcmart v1-29cm	contracts/collector/v1- {site}/	크롤러가 상품 dict를 만드는 즉시 (<code>contract_validation.validate_items</code>)	site별 구조 — 위 품은 리에 외되 error 록
v1-unified	contracts/collector/unified/	비교가 필요할 때만, <code>normalizer.normalize()</code> 통과 후	ABC 트/29 같은 로 비 능 측 (Pyth 등)
v1 (운영)	contracts/collector/v1/	— 아직 미구현 (Go Collector + RabbitMQ 전 제)	prov 포함 저장 지금 재는 abcm 본 di Adap 직접

공통 규칙

규칙	내용
<code>additionalProperties: false</code>	스키마에 없는 필드가 하나라도 있으면 그 상품은 검증 실패 — 새 필드를 크롤러에 추가하면 스키마도 같이 고쳐야 한다
<code>price</code> 문자열	"19,000원" 형태 고정 (<code>^\$ ^[0-9,]+원\$</code>) — 숫자·통화코드 분리하는 운영 계약(v1)의 몫
<code>verification</code> 서브스키마	두 site 계약 모두 동일한 <code>status</code> / <code>compared_fields</code> / <code>differences</code> / <code>json_source_url</code> / <code>html_source_url</code> / <code>verified_at</code> 구조 공유

v1-unified 핵심 필드

두 site 결과를 하나의 배열로 다루기 위한 최소 공통 집합. site 전용 리뷰 필드는 없으면 null로 채운다 (ABC마트: detail_scores/review_color/is_best, 29CM: user_size/partner_comment/size_survey/is_blind).

필드	타입	비고
source_product_id, title, brand, link, site	string	필수
price, price_original	string	필수, "원" 문자열 유지
discount_percent, rating, review_count	number null	필수(값은 null 허용)
image_url, images[], color, style_code	string	필수
category, category_path, in_stock	string / bool null	필수
options.colors[], options.sizes[]	string[]	29CM sizes는 options[].value에서 정규식으로 KR 사이즈만 추출
reviews[]	object[]	공통 6필드 + site 전용 필드 (null 채움)

6. 실행 예시 scripts/run_unified_crawl.py 구두

두 크롤러를 동시 실행 → site별 contract 검증 → normalize → 하나의 output/unified_*.json으로 저장한 실제 결과에서 상품 1건씩 발췌 (리뷰 배열은 길이만 표시).

ABCMART · 이브 웨지 3

```
{
  "price": "19,000원", "discount_percent": 61,
  "category": "웨지", "in_stock": true,
  "rating": 4.5, "review_count": 13,
  "options": {
    "colors": ["BLACK"],
    "sizes": ["225", "230", "235", "240", "245", "250"]
  },
  "reviews": "[13개, 전량 수집]"
}
```

29CM · BELLA SLINGBACK

```
{
  "price": "119,280원", "discount_percent": 29,
  "category": "플랫", "in_stock": true,
  "rating": 4.5, "review_count": 766,
  "options": {
    "colors": ["BLACK"],
    "sizes": ["230", "235", "240", "245", "250"]
  },
  "reviews": "[716개, 전량 수집]"
}
```

review_count ≠ len(reviews)일 수 있음 — 29CM 리뷰 API가 응답하는 count(766)와 실제로 페이지네이션으로 끝까지 받아지는 results 총량(716)이 다르다. 직접 API를 마지막 페이지까지 호출해 확인한 값이라 우리 쪽 페이지네이션 버그가 아니라 29CM API 자체의 카운트 오차다. review_count는 API가 보고하는 값을 그대로 두고, reviews[]는 실제로 받아올 수 있었던 만큼만 채운다.

키워드 "구두" · 판매처당 5~10개 기준 실행에서 **contract 위반 0건**, **unified-product.schema.json 전량 통과**를 확인했다.

7. 남은 논의거리

구조 ABC마트 카테고리 **crawl_category**)은 아직 HTML 전용 — 키워드 검색처럼 JSON 우선 + 검증용 크롤링(브라우저로 리팩토링할지

범위 운영 **contracts/collector/v1** (Go Collector 전제)이 여전히 미구현 — 지금 DB 적재는 v1-abcmart 원본을 Adapter가 직접 변환해 우회 중

범위 §3.3 자연어 검색 → 추천 파이프라인(dev-jw)은 이번에 이식하지 않음 — Spring 쪽 API 자체가 달라서 별도 작업 필요

속도 리뷰 전량 수집이 기본값이 되면서 리뷰 많은 상품(700+)은 상품당 페이지 요청이 늘어남 — 대량 실행 전 요청 예산/시간 상한 정책 필요